Automatic Domain Randomization in a sim-to-sim Reinforcement Learning scenario

1st Niccolò Malgeri
Polytechnic univerisity of Turin
Turin, Italy
s345370@studenti.polito.it

***Abstract*—In this work several experiments are conducted to explore the effect of Domain Randomization (DR) for achieving policy transfer in a sim-to-sim Reinforcement Learning context. The first part aims to understand the advantages of using a form of DR called Uniform Domain Randomization (UDR) over not using it; the second part regards the comparison between Uniform Domain Randomization and Automatic Domain Randomization, a recent DR approach that showed promising results in a sim-to-real scenario. It will be seen that the performance reached by using UDR are better than the ones obtained without it, and that ADR is generally an additional enhancement with respect to UDR.**

I. Introduction

Making a robotic system learn specific behaviors, such as moving objects or navigating 3D spaces, is a challenging task. Using Reinforcement Learning, which is based on doing actions in a trial-and-error fashion and seeing the effects of them to improve the next decisions, would require the robot to try possibly dangerous moves, that could severely damage its hardware and make the whole training process economically expensive and time-consuming. An alternative and less expensive approach is to perform training in a simulated environment using dynamics parameters that are similar to the real ones, and then test the performances in the real world (*sim-to-real* instead of *real-to-real*); in Reinforcement Learning such technique is called ***policy transfer***. To be effective, the simulated version should be as close as possible to the real one (in terms of dynamic parameters), which is practically infeasible even with modern simulators; this phenomenon is called ***sim-to-real gap***. Fortunately, reducing this gap is possible by randomizing the dynamics parameters during training, an approach called ***Domain Randomization*** (***DR***); with it, a robust policy can be learned and correctly transferred to a real world scenario. To assess the efficacy of this method, the Mujoco Hopper Environment is considered, and two randomization algorithms are compared: ***Uniform Domain Randomization*** (***UDR***) and ***Automatic Domain Randomization*** (***ADR***). The latter one, in which the randomization distribution is not fixed but changes dinamically to better generalize the task, lets the model obtain a better performance on the target environment. In the experiments, UDR and ADR are first used in a simple environment, then the comparison is shifted to a more complex one. Since evaluating the results in the real world was not possible, the sim-to-real context is reproduced as a sim-to-sim scenario, with the test simulated environment's parameters being shifted with respect to the training ones. For all the experiments ***PPO*** (***Proximal Policy Optimization***) is used for policy training.

II. Theoretical background

As briefly stated in Section I, **Domain Randomization** (***DR***) is a technique used in Reinforcement Learning to reduce the sim-to-real gap between a simulated environment and its real-world counterpart; in DR when training a policy in simulation, rather than using fixed dynamics parameters, these parameters are sampled from a predefined distribution at the beginning of each new training episode. This variability lets the learned policy generalize better to unseen scenarios, making the transfer to the real environment less problematic, since it is interpreted as an additional instance of the simulated environment with different values for the parameters. Speaking more in general, especially in computer vision tasks, DR can be used to randomize visual aspects of the environment such as textures, shapes and colors (*DR for appearance*); however, in the context of this work, DR is applied only to dynamics parameters.

A. Uniform Domain Randomization

In ***Uniform Domain Randomization*** (***UDR***), at the start of each training episode the environment parameters λ are sampled from a uniform distribution:

$$\lambda_i \sim \mathcal{U}(a_i, b_i),\ \forall i \tag{1}$$

where a_i and b_i are fixed values. Despite its semplicity, UDR has some weaknesses given by the fact that the distribution ranges are manually chosen and fixed for the whole training, leading to possible poor results if the bounds a_i and b_i are not selected properly. In particular:

- if the randomization range is too small, then during training only few environments will be explored, causing poor generalization and possibly overfitting to the simulated environment;
- if the randomization range is too wide, then the trained policy might learn a suboptimal strategy that performs decently for several different environments, but without excelling on any of them. Another issue of overgeneralization is learning to behave on unrealistic environments, without effectively bringing any improvement during the test phase;

B. Automatic Domain Randomization

One possible solution to overcome UDR drawbacks is to make the randomization ranges dynamic, adjusting them based on how well the learning policy performs with the current parameters distribution. That's exactly what is done in ***Automatic Domain Randomization*** (***ADR***), an algorithm created by *OpenAI* [1] in 2019. A clearer and more detailed explanation follows in Section III.

C. Proximal Policy Optimization

PPO (***Proximal Policy Optimization*** is a policy gradient algorithm belonging to the subset of ***Actor-Critic*** algorithms, which rely on two estimators:

- a **critic**, which performs policy evaluation by estimating the Value function (or the Action-Value function/Advantage function);
- an **actor**, which learns a stochastic policy based on the critic's evaluations;

In PPO, the actor is trained to maximize the expected cumulative reward, avoiding unbalanced gradient updates that can either cause instability (if they are too large) or significantly slow down the learning process (if they are too small).

III. Related Works: Solving Rubik's cube with a robot hand

As mentioned in Section II-B ADR was first introduced by OpenAI and used to make a robotic hand learn how to solve a Rubik's cube. By combining a control policy, a vision model and a LSTM (*Long Short Term Memory*) mechanism they managed to reach extraordinary results on a such high level complexity task in the real world, after a long training process in simulation. Their algorithm (**Algorithm 1**) works as follows:

1. start from dynamics parameters $\lambda \in \mathbf{R}^d$, where d is the number of environment parameters, an initial distribution P_{ϕ_0} with boundaries ϕ_{0_L} and ϕ_{0_H}, and two fixed performance thresholds t_L (lower bound) and t_H (upper bound);
2. on each training iteration:
   - 2.1 select randomly a parameter λ_i and one between the upper and lower bound of the distribution (a process called *boundary sampling*);
   - 2.2 assign to λ_i the chosen boundary value, sample the rest of the parameters from the current distribution P_ϕ (note that this is a multivariate distribution over all parameters) and save the model performances with such setup in a buffer $\mathcal{D}_i^L$ (or $\mathcal{D}_i^H$);
   - 2.3 if the buffer is full, compute the average performance $\bar{p}$ using the values stored in it. If $\bar{p} > t_H$ widen the randomization range for λ_i, shrink it instead if $\bar{p} < t_L$;

To quantify the amount of boundary expansion, in [1] the researchers defined the ***ADR entropy***:

$$\mathcal{H}(d) = \frac{1}{d}\sum_{i=1}^{d}\log(\phi_i^H - \phi_i^L) \tag{2}$$

Algorithm 1 ADR

Require: ϕ_0
Require: $\{\phi_i^L, \phi_i^H\}_{i=1}^d$
Require: m, t_L, t_H where $t_L < t_H$
Require: Δ
$\phi \leftarrow \phi^0$
repeat:
$\lambda \sim P_\phi$
$i \sim U\{1, \ldots, d\}$, $x \sim U(0, 1)$
if $x < 0.5$ **then**
    $D_i \leftarrow D_i^L$, $\lambda_i \leftarrow \phi_i^L$
else
    $D_i \leftarrow D_i^H$, $\lambda_i \leftarrow \phi_i^H$
$p \leftarrow$ EvaluatePerformance(λ)
$D_i \leftarrow D_i \cup \{p\}$
if Length$(D_i) \geq m$ **then**
    $\bar{p} \leftarrow$ Average(D_i)
    Clear(D_i)
    if $\bar{p} \geq t_H$ **then**
        $\phi_i \leftarrow \phi_i + \Delta$
    else if $\bar{p} \leq t_L$ **then**
        $\phi_i \leftarrow \phi_i - \Delta$
until training is complete

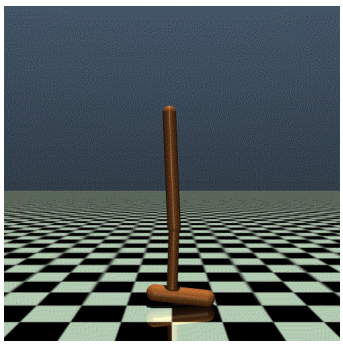


Fig. 1: Gym Hopper environment on MuJoCo

which represents the average length of the randomization range over all the environment parameters.

IV. Experimental setup

A. MuJoCo and the Gym Hopper Environment

MuJoCo (*Multi-Joint Dynamics with Contact*) is a physics simulation engine used for research and development in robotics, animation, biomechanics and graphics. *Gym* is instead a Python library providing a lot of different Reinforcement Learning environments that can be simulated using MuJoCo. One of them is the **Hopper** (Fig. 1), a two-dimensional one-legged figure composed of four body parts: torso, thigh, leg and a single foot. The goal is making the Hopper jump in the right direction, by applying torques to the three available hinges located between the masses of the body parts. A description of the main features of the Hopper Environment follows.

TABLE I: Hopper action space's dimensions

Num	Action	Control Min	Control Max	Joint	Unit
0	Torque applied on the thigh rotor	-1	1	hinge	torque (N·m)
1	Torque applied on the leg rotor	-1	1	hinge	torque (N·m)
3	Torque applied on the foot rotor	-1	1	hinge	torque (N·m)

TABLE II: Hopper Observation Space's Dimensions

Num	Observation	Min	Max	Joint	Unit
0	z-coordinate of the top (height of hopper)	-Inf	Inf	slide	position (m)
1	angle of the top	-Inf	Inf	hinge	angle (rad)
2	angle of the thigh joint	-Inf	Inf	hinge	angle (rad)
3	angle of the leg joint	-Inf	Inf	hinge	angle (rad)
4	angle of the foot joint	-Inf	Inf	hinge	angle (rad)
5	velocity of the x-coordinate of the top	-Inf	Inf	slide	velocity (m/s)
6	velocity of the z-coordinate (height) of the top	-Inf	Inf	slide	velocity (m/s)
7	angular velocity of the angle of the top	-Inf	Inf	hinge	angular velocity (rad/s)
8	angular velocity of the thigh hinge	-Inf	Inf	hinge	angular velocity (rad/s)
9	angular velocity of the leg hinge	-Inf	Inf	hinge	angular velocity (rad/s)
10	angular velocity of the foot hinge	-Inf	Inf	hinge	angular velocity (rad/s)

- **Action Space**: the action space is 3-dimensional, continuous and bounded in $[-1.0, 1.0]$ (Table I);
- **Observation Space**: the observation space is 11-dimensional, continuous and unbounded (Table II);
- **Reward Function**: the reward function consists of three terms:
  - *healthy reward*: gives a fixed positive value if the Hopper is still alive;
  - *forward reward*: gives a positive number if the Hopper moved in the right direction;
  - *control cost*: a cost term penalizing excessively large actions;

  The overall reward function is thus defined as:

$$\text{reward} = \text{healthy\_reward} + \text{forward\_reward} - \text{ctrl\_cost} \quad (3)$$

- **Masses**: the torso, thigh, leg and foot masses (Kg) are respectively $2.53429174, 3.92609982, 2.71433605$ and 5.08938010.

For the experiments, two versions of the Hopper environment are used:

- *source environment*: it's the default Hopper Environment;
- *target environment*: it's obtained by shifting the torso mass from 2.53429174 to 3.53429174 Kg; this makes the *target* environment more representative of the the *source*'s real-world counterpart, since an actual real-world scenario was not accessible;

The performances are evaluated on three policy transfer scenarios:

- **source to source**: policy trained on *source* and tested on *source*

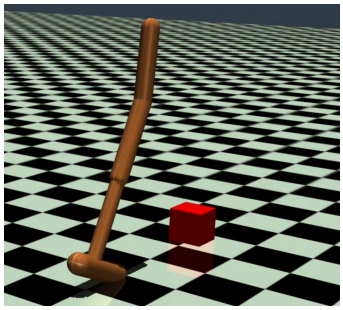


Fig. 2: Gym Hopper environment with an obstacle on MuJoCo

TABLE III: Hopper + obstacle Observation Space's Dimensions

Num	Observation	Min	Max	Joint	Unit
0	z-coordinate of the top (height of hopper)	-Inf	Inf	slide	position (m)
1	angle of the top	-Inf	Inf	hinge	angle (rad)
2	angle of the thigh joint	-Inf	Inf	hinge	angle (rad)
3	angle of the leg joint	-Inf	Inf	hinge	angle (rad)
4	angle of the foot joint	-Inf	Inf	hinge	angle (rad)
5	velocity of the x-coordinate of the top	-Inf	Inf	slide	velocity (m/s)
6	velocity of the z-coordinate (height) of the top	-Inf	Inf	slide	velocity (m/s)
7	angular velocity of the angle of the top	-Inf	Inf	hinge	angular velocity (rad/s)
8	angular velocity of the thigh hinge	-Inf	Inf	hinge	angular velocity (rad/s)
9	angular velocity of the leg hinge	-Inf	Inf	hinge	angular velocity (rad/s)
10	angular velocity of the foot hinge	-Inf	Inf	hinge	angular velocity (rad/s)
11	x-coordinate of obstacle's center	-Inf	Inf	slide	position (m)

- **target to target**: policy trained on *target* and tested on *target*
- **source to target**: policy trained on *source* and tested on *target*. This scenario is the most important one since it represents, better than the other two, how well the trained policy can perform on a different and possibly never seen environment.

B. Hopper Environment with an obstacle

To make the task more challenging and to further demonstrate the advantages of DR techniques for sim-to-real/sim-to-sim policy transfer, the original Hopper environment was modified to include an obstacle, which is a squared block of size $0.1\text{x}0.1\text{x}0.1m^3$ (Fig 2). The x-coordinate of the obstacle's center is also included in the observation space, which becomes now 12-dimensional (Table III). The choice of making the Hopper aware of the obstacle's position is due to avoiding overcomplicating the new task, but still keeping it difficult enough. In this modified Hopper environment, which will be called from now on *HopperObs*, the reward function used for training is slightly changed by adding an extra term obs_reward to give even more value to the positions beyond the obstacle one:

$$\begin{cases} \text{obs\_reward} = \text{hop\_pos\_x}, \text{hop\_pos\_x} > \text{obs\_pos\_x} \\ 0, \text{otherwise} \end{cases} \quad (4)$$

For the experiments, the masses setup for *source* and *target* environments remained the same; regarding the obstacle, its position is set to 2.5 for the *target* and 3.5 for the *source*. Since a more complex environment is introduced primarily to investigate more in depth the capabilities of ADR with respect to UDR, only the **source to target** scenario is considered, as it best highlights the advantages of DR techniques.

C. PPO and ADR configuration

As briefly stated in Section I, PPO is the method chosen for policy training; in particular, the implementation used is the one provided by the Python library for Reinforcement Learning *Stable Baselines3*. Since the results are compatible with what was expected, all the default parameters are left unchanged. To compute the average performance during training with ADR, the method *evaluate_policy()* from Stable Baselines3 is exploited, with a fixed number of test episodes set to 50. Regarding the buffers $\mathcal{D}_i^L$ and $\mathcal{D}_i^H$, the chosen size m is 10 for all dynamics parameters. Lastly, to reduce or expand the randomization ranges, rather than using the "shrinking/expanding factor" Δ as the original implementation, the boundaries change according to this simple rule:

- **shrink the boundary**: upper bound*0.95, lower bound*1.05;
- **expand the boundary**: upper bound*1.05, lower bound*0.95;

V. Test results

TABLE IV: Hopper Source to Source

Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	1519.27 ± 143.36
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$	N/A	1312.65 ± 230.28
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$	1400,800	1190.23 ± 114.84

TABLE V: Hopper Source to Target

Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	1090.15 ± 78.91
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$	N/A	1499.03 ± 168.22
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$	1200,300	914.05 ± 42.04
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$	1400,800	1272.17 ± 106.63

TABLE VI: Hopper Target to Target

Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	1692.53 ± 11.40
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$	N/A	1335.86 ± 147.02
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$	1400,800	1490.80 ± 434.39

TABLE VII: HopperObs Source to Target (fixed obstacle + normal reward on test)

Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	532.15 ± 45.78
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	N/A	683.00 ± 204.71
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	900,300	1100.74 ± 100.89

TABLE VIII: HopperObs Source to Target (fixed obstacle + obs reward on test)

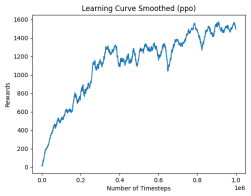
Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	573.03 ± 75.09
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	N/A	854.16 ± 382.44
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	900,300	1709.22 ± 229.54

TABLE IX: HopperObs Source to Target (moving obstacle + normal reward on test)

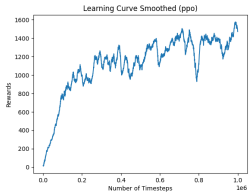
Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	532.15 ± 45.78
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	N/A	800.09 ± 302.24
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	900,300	1034.53 ± 125.92
PPO + UDR	$[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$	N/A	670.09 ± 120.98
PPO + ADR	$[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$	900,300	977.10 ± 72.31

TABLE X: HopperObs Source to Target (moving obstacle + obs reward on test)

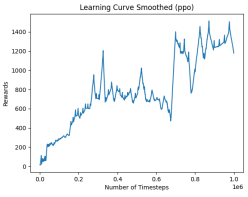
Method	DR Range	t_L, t_H	Mean Reward ± Std Dev
PPO	N/A	N/A	570.57 ± 70.16
PPO + UDR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	N/A	1096.10 ± 555.34
PPO + ADR	$[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$	900,300	1536.04 ± 307.76
PPO + UDR	$[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$	N/A	822.93 ± 195.47
PPO + ADR	$[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$	900,300	1398.89 ± 160.46



(a) PPO

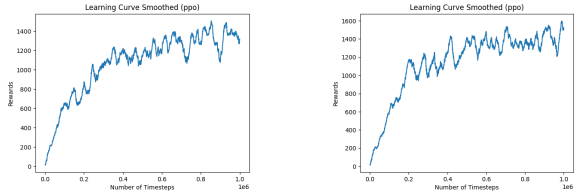


(b) PPO + UDR



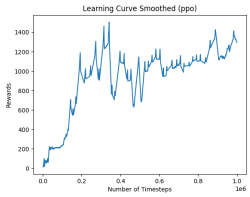
(c) PPO + ADR

Fig. 3: Hopper Source mean reward over 1000000 training timesteps (DR range = $[\lambda * 0.4, \lambda * 1.2]$, t_L, t_H = (1400,800))



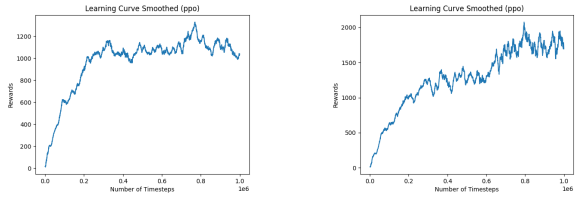
(a) PPO

(b) PPO + UDR



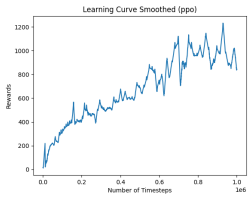
(c) PPO + ADR

Fig. 4: Hopper Target mean reward over 1000000 training timesteps (DR range = $[\lambda * 0.4, \lambda * 1.2]$, t_L, t_H = (1400,800))



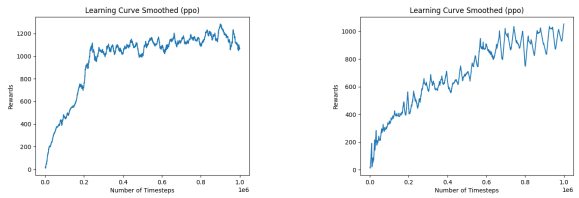
(a) PPO

(b) PPO + UDR



(c) PPO + ADR

Fig. 5: HopperObs Source mean reward over 1000000 training timesteps (DR range = $[\lambda * 0.4, \lambda * 1.2]$, $\lambda' \pm 0.5$, t_L, t_H = (900,300))



(a) PPO + UDR

(b) PPO + ADR

Fig. 6: HopperObs Source mean reward over 1000000 training timesteps (DR range = $[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$, t_L, t_H = (900,300))

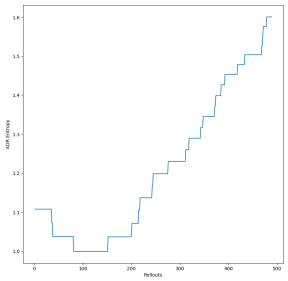


Fig. 7: Hopper Target ADR entropy over 500 rollouts (1000000 timesteps) (DR range = $[\lambda * 0.4, \lambda * 1.2]$, t_L, t_H = (1400,800))

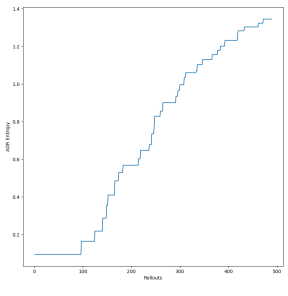


Fig. 8: HopperObs Source ADR entropy over 500 rollouts (1000000 timesteps) (DR range = $[\lambda * 0.8, \lambda * 1.1]$, $\lambda' \pm 0.5$, t_L, t_H = (900,300))

VI. Analysis of Results

A. Source to Source

In the Hopper Source-to-Source test (Table IV), vanilla PPO achieved the best results, followed by PPO with UDR and then PPO with ADR. This is because, without DR, the policy focuses solely on optimizing for the specific environment seen during training, effectively overfitting to it. In contrast, both UDR and ADR encourage the policy to learn a more generalizable behavior that can perform decently across multiple environments, at the cost of a slightly lower performance on the training environment itself.

PPO with ADR performs slightly worse than PPO with UDR likely due to the simplicity of the task. In simpler environments, aggressive domain adaptations, such as those introduced by ADR, can lead to the exploration of unrealistic and unnecessary variations, which may never be encountered in the test environment. This can reduce performance compared to less aggressive DR approaches like UDR, which provides a more stable and controlled training distribution, as can be seen in Fig.3.

B. Target to Target

In the Hopper Target-to-Target test (Table VI) the results are fairly similar to those from the Source-to-Source test; this is an expected outcome since the test environment is equal to the training one. Apparently, PPO + ADR seems to reach a better mean reward, higher than the one reached by PPO + UDR, at the cost of a higher variance, which makes ADR again slightly worse than UDR.

C. Source to Target - Hopper

In the Hopper Source-To-Target test (Table V), as expected, PPO + UDR performed noticeably better than vanilla PPO, obtaining an average reward discrepancy of almost 500, and proving the robusntess and the generalization capabilities of a DR-trained policy. With performance thresholds of 1200 and 300, PPO + ADR strangely performed worse even than vanilla PPO, probably because a lower threshold that does not require a high amount of average performance to be surpassed, in simple environments, can make the policy focus more on difficult environments, moving "away" from simpler ones without learning an effective strategy for them. After increasing the lower threshold to 800 and the higher threshold to 1400, PPO + ADR still remains less performant with respect to PPO + UDR, but now it is able to outclass vanilla PPO with an average reward discrepancy of about 200. Since this second choice of the thresholds lead to better results, the same values are used in both Source-to-Source and Target-To-Target scenarios.

The effect of increasing the lower performance threshold is clearly visible in Fig. 7, which shows the ADR entropy throughout the training on the Hopper Target environment; since in the first part of the training the policy is unable to reach an average reward of 800 (Fig. 4c), there is a reduction of the randomization ranges, which consequently reduces the entropy. As the learning progresses, the policy becomes able to surpass the minimum performance requirements, causing the randomization ranges to expand and, consequently, the entropy to increase.

D. Source To Target - HopperObs

In the HopperObs Source-to-Target tests the results are more promising. With a fixed obstacle on Target environment, vanilla PPO and PPO + UDR are completely unable to jump the obstacle, crashing next to it. This is probably due to an unfortunate positioning of the obstacle, that is either too far away from the starting point and consequently never explored during training, or its position makes the initial jump difficult or even not possible. Instead, PPO + ADR reaches an incredibly high average reward of 1700 with the HopperObs reward and 1100 with the original one (Tables VII and VIII), correctly jumping the obstacle on both cases. To further investigate, vanilla PPO and its DR versions are tested with moving obstacles, and again ADR reaches the highest performance with both the default and the HopperObs reward (Tables IX and X), proving the generalization capabilities of ADR on tasks with a significant difficulty.

To prove instead that a bad choice for the distribution ranges severly conditions UDR's performances, but does not affect ADR at all, another test with a moving obstacle is done, in which the boundaries for dynamics parameters are shifted from $[\lambda * 0.4, \lambda * 1.2]$ to $[\lambda * 0.8, \lambda * 1.1]$. By looking again at Tables IX and X, it can be noticed that the performances of PPO + UDR are, as expected, worse than PPO + ADR, which is able to overcome the initial bad distribution choice and reach a higher average mean reward as well as a lower variance. The stability of ADR in HopperObs (Fig. 5c and 6b) rather than in the default Hopper environment strengthens more the idea that, for more challenging tasks, it is a better DR algorithm than UDR.

VII. Conclusions

References

[1] OpenAI, Ilge Akkaya, Marcin Andrychowicz, Maciek Chociej, Mateusz Litwin, Bob McGrew, Arthur Petron, Alex Paino, Matthias Plappert, Glenn Powell, Raphael Ribas, Jonas Schneider, Nikolas Tezak, Jerry Tworek, Peter Welinder, Lilian Weng, Qiming Yuan, Wojciech Zaremba, and Lei Zhang. *Solving Rubik's Cube with a Robot Hand. CoRR*, vol. abs/1910.07113, 2019. http://arxiv.org/abs/1910.07113